

API Bug Validation Test Plan

High AsyncNetworkIO and Memory Usage
When Loading Customer via Admin API

Document Version	1.0
Prepared by	Jeevan Paapagaari, Senior QA Engineer
Input	
Date	
Status	Ready To Use
Environment	To Be Filled While Executing
Feature Branch Base URL	To Be Filled While Executing
Dev Master Base URL	To Be Filled While Executing
Stage5 Base URL	To Be Filled While Executing

Table of Contents

- 1. Bug Summary & Scope
- 2. Root Cause Analysis (QA Perspective)
- 3. Test Objectives
- 4. Test Scenarios & Test Cases
- 5. Test Data Requirements
- 6. API Request & Response Validation Checklist
- 7. Tools & Frameworks
- 8. Entry & Exit Criteria
- 9. Risk & Mitigation
- 10. Defect Reporting Template
- 11. Appendix — Evidence Capture Checklist

1. Bug Summary & Scope

Bug Details

Bug ID	EDC-42653
Impact Analysis Ticket	EDC-42654
Title	High AsyncNetworkIO and memory usage when loading customer via Admin API
Severity	High (performance degradation impacting production Admin API)
Priority	High (active regression; tagged "FB - Testing in Progress")
Status	Testing in Progress
Impacted Area	Administration API — Customer & Study endpoints

Core Engineering Change

The fix moves CustomerDTO construction into the Entity Framework (EF) query layer so that EF generates a projection SQL SELECT (only required columns) rather than loading full entity graphs into memory. This eliminates over-fetching as the root cause of high AsyncNetworkIO waits and elevated memory/CPU.

Affected API Endpoints

#	Method	Endpoint	Version	ID Type
1	GET	/api/v1/customer/{customerId}	v1	long (numeric)
2	GET	/api/v1/customer/{globalCustomerId}	v1	guid
3	GET	/api/v1/customer/study/{studyId}	v1	long (numeric)

Environments in Scope

Environment	Purpose	Execution Level
Feature Branch (FB)	Smoke + correctness — verify refactor does not change API behavior	Functional, auth, quick perf sanity
Dev Master	Full functional + regression + baseline comparisons	All FB tests + edge cases + concurrency + perf/memory baseline
Stage	Production-like confidence with realistic dataset	Functional on representative large data + concurrency/perf + memory stability

Impacted Services & Downstream Consumers

- Admin API — direct owner of these endpoints
- Admin UI / Internal Tooling — any tools consuming 'Get Customer' or 'Get Study' API calls
- Downstream systems — any service relying on the Customer or Study response contract
- Entity Framework query layer — refactored data access; potential blast radius for serialization and query translation

2. Root Cause Analysis (QA Perspective)

Identified Failure Point

The defect lies in the data access / ORM layer, not in the HTTP layer, business logic, or auth stack. Specifically:

- EF was loading full entity graphs (all columns, all navigations) and performing DTO projection in-memory (LINQ-to-Objects), rather than generating a projected SQL query.
- This caused excessive AsyncNetworkIO waits (large row transfers from DB), elevated heap allocation (full entities held in memory before projection), and increased CPU (in-process mapping of unneeded data).

QA-Identified Failure Surface

Layer	Risk Type	Detail
Response data	Field regression	EF projection changes may accidentally include or drop fields from the DTO → silent contract break
Null / Empty relation handling	Serialization	Customers with 0 studies or minimal data may serialize differently when EF projection returns null navigations vs empty collections
Error responses	EF translation failure	A flawed LINQ projection that EF cannot translate will throw InvalidOperationException at runtime — surfacing as 500s
Auth behavior	Unchanged invariant	Query-layer change should not affect middleware auth; must be verified explicitly
Performance	Regression vs improvement	If projection is incomplete, the fix may not fully eliminate AsyncNetworkIO waits

3. Test Objectives

Primary Objective

Validate that the EF query projection fix resolves the AsyncNetworkIO and memory regression without altering any API response contract for the three affected endpoints across all environments.

Secondary Objectives

#	Objective	Category
1	Confirm response schema, status codes, and data content are identical to pre-fix baseline for all happy-path inputs	Regression / Contract
2	Confirm negative/error paths (400, 404, 401, 403) behave identically to pre-fix baseline	Error Handling
3	Measure and document response-time improvement (AsyncNetworkIO symptom reduction)	Non-functional — Performance
4	Confirm no memory growth or leak pattern under sustained load on the fixed build	Non-functional — Memory
5	Validate that auth/authorization invariants are unchanged (EDC-42654 states "no security impact")	Security posture
6	Validate EF query translation does not throw runtime errors for any edge-case input	Stability
7	Verify no new 500 errors or EF/serialization exceptions introduced in logs	Regression
8	Validate downstream consumers (Admin UI) are not broken by any response contract change	Integration Regression

4. Test Scenarios & Test Cases

Category A — Bug Reproduction

Purpose: Prove the original defect is observable on the pre-fix build. Execute these on the baseline (unfixed) build to establish evidence the bug was real.

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-A-001	Reproduce high response time on large customer (customerId)	GET	/api/v1/customer/{customerId}	1. Pre-fix build deployed. 2. valid admin token. 3. large customer seeded (many studies/domains)	customerId = large customer ID	1. Authenticate. 2. Call endpoint with large customerId. 3. Record response time and observe AsyncNetworkIO in telemetry.	1. Response time noticeably higher than acceptable SLA. 2. AsyncNetworkIO metric elevated in monitoring		
TC-A-002	Reproduce high memory usage during burst on large customer (studyId)	GET	/api/v1/customer/study/{studyId}	1. Pre-fix build 2. Valid admin token 3. Large study seeded	studyId = large study ID	1. Authenticate. 2. Fire 20 sequential calls. 3. Observe memory trend in monitoring.	1. Memory climbs during burst. 2. Does not recover promptly		

Category B — Fix Validation (Confirms Resolution on Fixed Build)

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-B-001	Valid minimal customer by customerId returns 200	GET	/api/v1/customer/{customerId}	Fixed build deployed Valid admin token Minimal customer seeded (0–1 studies)	customerId = minimal customer ID	1. Authenticate. 2. Call endpoint. 3. Assert status. 4. Assert schema.	HTTP 200; response body matches expected CustomerDTO schema; all required fields present		

API Bug Validation Test Plan

TC-B-002	Valid medium customer by customerId returns 200	GET	/api/v1/customer/{customerId}	Fixed build; valid admin token; medium customer seeded	customerId = medium customer ID	1–4 as TC-B-001	HTTP 200; full and correct response		
TC-B-003	Valid large/complex customer by customerId returns 200	GET	/api/v1/customer/{customerId}	Fixed build; valid admin token; large customer seeded (many studies, domains, associations)	customerId = large customer ID	1–4 as TC-B-001	HTTP 200; full and correct response; response time improved vs pre-fix baseline		
TC-B-004	Valid minimal customer by globalCustomerId returns 200 and matches customerId response	GET	/api/v1/customer/{globalCustomerId}	Fixed build; valid admin token; same minimal customer has known globalCustomerId	globalCustomerId = minimal customer GUID	1. Call endpoint with globalCustomerId. 2. Compare response body with TC-B-001 for same entity.	HTTP 200; response content semantically matches /customer/{customerId} response for same entity		
TC-B-005	Valid large customer by globalCustomerId returns 200	GET	/api/v1/customer/{globalCustomerId}	Fixed build; valid admin token; large customer GUID known	globalCustomerId = large customer GUID	1–4 as TC-B-004	HTTP 200; full and correct response		
TC-B-006	Valid minimal study by studyId returns 200	GET	/api/v1/customer/study/{studyId}	Fixed build; valid admin token; minimal study seeded	studyId = minimal study ID	1. Authenticate. 2. Call endpoint. 3. Assert 200 and schema.	HTTP 200; response matches expected Study/CustomerDTO schema		
TC-B-007	Valid large/complex study by studyId returns 200	GET	/api/v1/customer/study/{studyId}	Fixed build; large study seeded	studyId = large study ID	1–3 as TC-B-006	HTTP 200; correct response; improved response time vs pre-fix baseline		
TC-B-008	Response equivalence — pre-fix vs fixed build (same input, same entity)	GET	All 3 endpoints	Both pre-fix and fixed builds are accessible; same test data inputs used on both	Large customer + large study IDs	1. Call each endpoint on pre-fix build. Save responses. 2. Call same endpoints on fixed build. 3. Diff response bodies field-by-field.	Zero schema or content differences; any differences documented and confirmed as acceptable/non-breaking per EDC-42654		

API Bug Validation Test Plan

TC-B-009	Customer with 0 studies returns 200 with empty/null studies collection (no serialization error)	GET	/api/v1/customer/{customerId}	Fixed build; customer seeded with no associated studies	customerId = customer with 0 studies	1. Call endpoint. 2. Assert 200. 3. Assert studies field is empty array or null. 4. Check logs for EF/serialization errors.	HTTP 200; no exceptions; studies collection is empty/null per API contract		
TC-B-010	Study with minimal configuration returns 200 (no EF translation error)	GET	/api/v1/customer/study/{studyId}	Fixed build; minimal study (few related objects) seeded	studyId = minimal study	1. Call endpoint. 2. Assert 200. 3. Check logs.	HTTP 200; no EF query translation errors in logs		

Category C — Boundary & Edge Cases

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-C-001	Non-existent numeric customerId returns 404	GET	/api/v1/customer/{customerId}	Fixed build; valid admin token	customerId = 999999999 (valid format, non-existent)	1. Call endpoint.	HTTP 404; error body follows standard error schema; no 500		
TC-C-002	Non-numeric customerId returns 400	GET	/api/v1/customer/{customerId}	Fixed build; valid admin token	customerId = "abc"	1. Call endpoint with string ID.	HTTP 400; validation error; no 500		
TC-C-003	Zero or negative customerId returns 400 or 404	GET	/api/v1/customer/{customerId}	Fixed build; valid admin token	customerId = 0, then -1	1. Call each. 2. Assert status.	HTTP 400 or 404 (per API contract); no 500		
TC-C-004	Malformed GUID for globalCustomerId returns 400	GET	/api/v1/customer/{globalCustomerId}	Fixed build; valid admin token	globalCustomerId = "not-a-guid"	1. Call endpoint.	HTTP 400; no 500		
TC-C-005	Non-existent GUID for globalCustomerId returns 404	GET	/api/v1/customer/{globalCustomerId}	Fixed build; valid admin token	globalCustomerId = 00000000-0000-0000-0000-000000000001	1. Call endpoint.	HTTP 404; no 500		

API Bug Validation Test Plan

					(valid format, non-existent)				
TC-C-006	Non-existent studyId returns 404	GET	/api/v1/customer/study/{studyId}	Fixed build; valid admin token	studyId = 999999999	1. Call endpoint.	HTTP 404; no 500		
TC-C-007	Customer in inactive/disabled state returns correct response	GET	/api/v1/customer/{customerId}	Fixed build; inactive customer seeded (if system models this state)	customerId = inactive customer	1. Call endpoint. 2. Verify response reflects correct state.	HTTP 200 (or 404 if inactive); correct state reflected; no 500		
TC-C-008	Customer with large/complex domain data returns correctly	GET	/api/v1/customer/{customerId}	Fixed build; customer with large domain data seeded	customerId = domain-heavy customer	1. Call endpoint. 2. Assert 200. 3. Assert all domain fields present and correct.	HTTP 200; all domain/association fields correctly projected; no truncation		
TC-C-009	List ordering stability (if order is part of contract)	GET	All 3 endpoints	Fixed build; large customer with multiple studies in known order	Same large customer/study IDs	1. Call endpoint 3 times. 2. Compare ordering of collections across calls.	Collection ordering is deterministic across calls (or confirmed as non-deterministic and documented)		

Category D — Authorization & Authentication

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-D-001	Call without any auth token returns 401	GET	/api/v1/customer/{customerId}	Fixed build; no token in request	Valid customerId	1. Call endpoint without Authorization header.	HTTP 401; no data leaked		
TC-D-002	Call with expired token returns 401	GET	/api/v1/customer/{customerId}	Fixed build; expired JWT/token	Valid customerId	1. Call endpoint with expired token.	HTTP 401		
TC-D-003	Call with valid token but insufficient role returns 403	GET	/api/v1/customer/{customerId}	Fixed build; token with non-admin role	Valid customerId	1. Call endpoint with under-privileged token.	HTTP 403		

API Bug Validation Test Plan

TC-D-004	Call without auth on globalCustomerId endpoint returns 401	GET	/api/v1/customer/{globalCustomerId}	Fixed build; no token	Valid GUID	1. Call without auth.	HTTP 401		
TC-D-005	Call without auth on study endpoint returns 401	GET	/api/v1/customer/study/{studyId}	Fixed build; no token	Valid studyId	1. Call without auth.	HTTP 401		
TC-D-006	Valid admin token returns 200 for all 3 endpoints	GET	All 3 endpoints	Fixed build; valid admin token	Minimal customer + minimal study	1. Call each with valid admin token.	HTTP 200 for all 3		
TC-D-007	Response does not expose additional fields vs pre-fix baseline (projection leak check)	GET	All 3 endpoints	Fixed build; pre-fix response saved	Same inputs as pre-fix	1. Compare response fields in fixed build vs pre-fix baseline.	No new fields present that were not in pre-fix response		

Category E — Error Handling

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-E-001	Any invalid input does not produce HTTP 500	GET	All 3 endpoints	Fixed build	Non-existent IDs, malformed IDs, zero IDs	1. Call with each invalid input type. 2. Assert no 500.	HTTP 400 or 404 as applicable; never 500		
TC-E-002	Error response body conforms to standard error schema	GET	All 3 endpoints	Fixed build; trigger 400 and 404	Invalid/missing IDs	1. Call with bad input. 2. Assert response body has required error fields.	Error response body matches documented error schema		
TC-E-003	Very large customer/study does not timeout or produce 504	GET	/api/v1/customer/{customerId}, /api/v1/customer/study/{studyId}	Fixed build; largest seeded customer/study	Large customer ID + large study ID	1. Call each. 2. Assert 200 within timeout threshold.	HTTP 200 within acceptable timeout; no 504		

API Bug Validation Test Plan

TC-E-004	No EF query translation exceptions in logs for any input	GET	All 3 endpoints	Fixed build; log aggregation accessible	All input categories across A–C	1. Execute test suite. 2. Review logs for InvalidOperationException, QueryTranslationException, or EF errors.	Zero EF query errors in logs correlated to these requests		
TC-E-005	No serialization errors in logs (null/empty navigation handling)	GET	All 3 endpoints	Fixed build; customers with 0 studies	Customer with no studies	1. Call endpoint. 2. Review logs for JsonSerializerException or similar.	Zero serialization errors in logs		

Category F — Contract Testing

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-F-001	Response Content-Type is application/json	GET	All 3 endpoints	Fixed build; valid admin token	Any valid ID	1. Call endpoint. 2. Assert Content-Type header present.	Content-Type: application/json; charset=utf-8 (or equivalent)		
TC-F-002	Response schema matches OpenAPI/documented contract	GET	All 3 endpoints	Fixed build; valid admin token; OpenAPI spec available	Valid IDs	1. Call each endpoint. 2. Validate response body against JSON Schema / OpenAPI spec.	No schema violations; all required fields present; no unexpected additional fields		
TC-F-003	customerId and globalCustomerId endpoints return same fields for the same entity	GET	/api/v1/customer/{customerId} and /api/v1/customer/{globalCustomerId}	Fixed build; customer with known both ID types	Same customer, both ID types	1. Call both endpoints. 2. Compare response schemas and content.	Response schemas are identical; content matches for same entity		
TC-F-004	Nested object fields (studies, domains) present and correctly typed	GET	/api/v1/customer/{customerId}	Fixed build; customer with studies and domains	Large customer ID	1. Call endpoint. 2. Assert nested arrays/objects are present and typed correctly.	All nested fields correctly typed; no type coercion issues introduced by EF projection		
TC-F-005	No CORS or unexpected header changes vs pre-fix baseline	GET	All 3 endpoints	Fixed build; pre-fix header	Same inputs	1. Compare response headers pre-fix vs fixed.	Response headers unchanged		

				baseline documented					
--	--	--	--	---------------------	--	--	--	--	--

Category G — Regression

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
TC-G-001	Other Admin API endpoints unaffected by this change	GET	Other Admin API endpoints (not in scope of EDC-42653)	Fixed build; valid admin token	Sample calls to adjacent endpoints	1. Call 2–3 adjacent Admin API endpoints. 2. Assert 200 and correct responses.	HTTP 200; no regressions introduced; confirms change is scoped correctly		
TC-G-002	Admin UI or downstream tool can still load customer data correctly	GET	/api/v1/customer/{customerId}	Fixed build; Admin UI / downstream tool accessible	Large customer	1. Load customer in Admin UI / consumer tool. 2. Verify data renders correctly.	Customer data renders correctly; no missing fields; no UI errors		
TC-G-003	No new EF query translation errors in logs across all tests	—	All 3 endpoints	Fixed build; full test run completed	All test inputs	1. Review aggregated logs after full test run.	Zero new EF, EntityFramework, or QueryTranslation errors in logs		
TC-G-004	No circular reference or serialization errors in logs	—	All 3 endpoints	Fixed build	All test inputs	1. Review logs for JSON circular reference errors or System.Text.Json exceptions.	Zero serialization errors		

Category H — Performance

TC ID	Scenario	Method	Endpoint	Preconditions	Test Data	Steps	Expected Result	Actual Result	Pass/Fail
-------	----------	--------	----------	---------------	-----------	-------	-----------------	---------------	-----------

API Bug Validation Test Plan

TC-H-001	Response time improvement — small customer (customerId)	GET	/api/v1/customer/{customerId}	Both pre-fix and fixed builds; minimal customer seeded	Minimal customer ID	1. Record response time on pre-fix build (5 calls, average). 2. Record on fixed build. 3. Compare.	Fixed build average $\leq$ pre-fix average; improvement documented		
TC-H-002	Response time improvement — large customer (customerId)	GET	/api/v1/customer/{customerId}	Both builds; large customer seeded	Large customer ID	1–3 as TC-H-001	Measurable improvement in fixed build; no degradation		
TC-H-003	Response time improvement — large study (studyId)	GET	/api/v1/customer/study/{studyId}	Both builds; large study seeded	Large study ID	1–3 as TC-H-001	Measurable improvement in fixed build		
TC-H-004	Burst test — 20–50 calls to large customer endpoint (customerId)	GET	/api/v1/customer/{customerId}	Fixed build; large customer seeded; monitoring accessible	Large customer ID	1. Fire 20–50 sequential/rapid calls. 2. Monitor response times. 3. Check for spikes, timeouts, or degradation.	No spike pattern; all calls return 200; P95 response time acceptable; no timeouts		
TC-H-005	Burst test — 20–50 calls to large study endpoint	GET	/api/v1/customer/study/{studyId}	Fixed build; large study seeded	Large study ID	1–3 as TC-H-004	Same as TC-H-004		
TC-H-006	AsyncNetworkIO metric reduced in monitoring during burst	GET	/api/v1/customer/{customerId}	Fixed build; monitoring/APM accessible	Large customer burst	1. Run TC-H-004. 2. Observe AsyncNetworkIO wait metric in monitoring.	AsyncNetworkIO metric reduced vs pre-fix baseline; no abnormal spikes		
TC-H-007	Sustained memory/CPU run — large customer (15 min)	GET	/api/v1/customer/{customerId}	Fixed build; monitoring accessible; large customer seeded	Large customer ID	1. Warm up (10 calls). 2. Run sustained calls for 15 minutes. 3. Observe CPU and memory trend.	CPU stable or improved vs baseline; memory does not continuously grow; GC recovery normal after load stops		
TC-H-008	Sustained memory/CPU run — large study (15 min)	GET	/api/v1/customer/study/{studyId}	Fixed build; monitoring; large study seeded	Large study ID	1–3 as TC-H-007	Same as TC-H-007		
TC-H-009	Concurrency — 10 parallel calls to large	GET	/api/v1/customer/{customerId}	Fixed build; large customer;	Large customer ID	1. Fire 10 parallel requests simultaneously. 2. Assert all return 200.	All 10 return HTTP 200; no 500s; no deadlocks; P99 latency within SLA		

API Bug Validation Test Plan

	customer (customerId)			concurrency tool ready		3. Assert no deadlocks. 4. Assert latency acceptable.			
TC- H- 010	Concurrency — 10 parallel calls to large study (studyId)	GET	/api/v1/customer/study/{studyId}	Fixed build; large study	Large study ID	Same as TC-H-009	Same as TC-H-009		
TC- H- 011	Service remains responsive to other calls during concurrency test	GET	Other Admin API endpoint	Fixed build; concurrency test running	Concurrent large calls + 1–2 unrelated Admin API calls	1. While TC-H-009 runs, call a separate Admin API endpoint. 2. Assert it returns 200.	Admin API not starved; concurrent load on these endpoints does not block other operations		

5. Test Data Requirements

Customer Test Profiles [Optional]

Profile	Description	Studies	Domains	Notes
Minimal	Few/no related entities	0–1	0	Tests null/empty EF projection handling
Medium	Typical production-like size	5–15	2–5	Baseline correctness
Large / Complex	Stress scenario	50+	Many	Primary perf test target; must be pre-seeded in each env
Inactive/Disabled	Unusual state	Any	Any	Tests state field projection correctness
Domain-heavy	Multiple domains + associations	Medium	10+	Validates domain projection completeness

Study Test Profiles

Profile	Description	Related Objects
Minimal	Few related objects (sites, forms)	1–2 sites, 1 form
Large	Many related objects	Many sites, forms, domains; references large customer

Identifier Test Matrix

ID Type	Valid	Invalid Format	Non-existent (valid format)
customerId (long)	Actual seeded IDs	"abc", 0, -1	999999999
globalCustomerId (guid)	Actual seeded GUIDs	"not-a-guid", "123"	00000000-0000-0000-0000-000000000001
studyId (long)	Actual seeded IDs	"abc", 0	999999998

Input Payloads (Request Examples)

Happy path — customerId
GET /api/v1/customer/10042
Authorization: Bearer <valid_admin_token>

Happy path — globalCustomerId
GET /api/v1/customer/3fa85f64-5717-4562-b3fc-2c963f66afa6
Authorization: Bearer <valid_admin_token>

Happy path — studyId
GET /api/v1/customer/study/20017
Authorization: Bearer <valid_admin_token>

Negative — non-numeric customerId
GET /api/v1/customer/abc
Authorization: Bearer <valid_admin_token>

Negative — malformed GUID
GET /api/v1/customer/not-a-guid
Authorization: Bearer <valid_admin_token>

Auth failure — missing token
GET /api/v1/customer/10042
(no Authorization header)

Environment Variables & Secrets Required

Variable	Description
ADMIN_API_BASE_URL	Base URL for Admin API (per environment)
ADMIN_VALID_TOKEN	Bearer token with admin role (valid, non-expired)
ADMIN_EXPIRED_TOKEN	Expired Bearer token (for TC-D-002)
ADMIN_NONPRIVILEGED_TOKEN	Bearer token without required admin role (for TC-D-003)
CUSTOMER_ID_MINIMAL	Seeded minimal customer long ID
CUSTOMER_ID_MEDIUM	Seeded medium customer long ID
CUSTOMER_ID_LARGE	Seeded large customer long ID
GLOBAL_CUSTOMER_ID_LARGE	Seeded large customer GUID
STUDY_ID_MINIMAL	Seeded minimal study long ID
STUDY_ID_LARGE	Seeded large study long ID

6. API Request & Response Validation Checklist

Status Code Verification

Input Condition	Expected Status
Valid ID, valid admin token	200 OK
Non-existent ID (correct format)	404 Not Found
Malformed ID (wrong format)	400 Bad Request
No auth token	401 Unauthorized
Valid token, insufficient role	403 Forbidden
Any error condition	Never 500 Internal Server Error

Response Body Schema Validation

- All fields present in the pre-fix CustomerDTO contract must be present and correctly typed in the fixed build.
- Nested arrays (e.g., studies list) must be an array (empty [] or populated), never null if the contract specifies array.
- Field types must match documented schema: customerId as integer, globalCustomerId as string (uuid), dates as ISO 8601 strings.
- No extra fields present that were absent in the pre-fix baseline (projection leak check — TC-D-007).
- Run response against JSON Schema / OpenAPI spec (if available) as a formal validation step.

Required Response Headers

Header	Expected Value
Content-Type	application/json (or application/json; charset=utf-8)
Cache-Control	Per API policy (verify unchanged)
X-Correlation-Id / X-Request-Id	Present if implemented

Response Time SLA

Scenario	Target (Fixed Build)
Small/minimal customer or study	≤ pre-fix baseline (improvement expected)
Large/complex customer or study	Measurable improvement vs pre-fix; absolute SLA to be confirmed with dev team
Burst (20–50 calls) P95	No spike above 2× single-call baseline
Concurrency (10 parallel) P99	No timeouts; all return 200

Note: Confirm exact SLA numbers with the engineering team based on pre-fix measurements.

Idempotency Check

All three endpoints are HTTP GET — they are naturally idempotent. Verify:

- Same input on multiple sequential calls returns identical response bodies (except non-deterministic fields like timestamps if applicable).
- No state mutation occurs as a side effect of any GET call.

Pagination / Filtering Behavior

- If the GET /api/v1/customer/study/{studyId} or customer endpoints support pagination parameters, verify these are unaffected by the EF projection change.
- If study lists within a customer response are paginated or filtered, verify ordering and completeness of results are unchanged.

7. Tools & Frameworks

Functional & Contract Testing

Tool	Use Case
Postman	Manual exploratory + automated collection runner for FB smoke; response equivalence diffs
JSON Schema Validator	Schema assertion for TC-F-002; integrate via RestAssured or standalone

Performance & Load Testing

Tool	Use Case
Postman Runner	Quick manual burst sanity (20–50 iterations) on FB

Monitoring & Observability [If Possible, Need to Check]

Tool	Use Case
APM (e.g., Application Insights, Datadog, New Relic)	Observe AsyncNetworkIO waits, memory trends, CPU during test runs
SQL Server Profiler / Query Store	Validate that new EF projection is generating the expected projected SQL (not SELECT *)
Log aggregation (e.g., Seq, Splunk, ELK)	Search for EF errors, serialization exceptions, 500-level log entries correlated to test runs

Mocking Strategy

- No mocking recommended for these tests — the bug is in the data access layer; tests must exercise real DB queries to validate EF projection correctness.
- If DB is unavailable in early FB environment, use WireMock or Mockoon only to stub downstream dependencies of the Admin API (not the DB itself).

8. Entry & Exit Criteria

Entry Criteria

#	Criterion
1	Fixed build successfully deployed to target environment (FB / Dev Master / Stage)
2	Admin API endpoints are reachable and returning responses (basic health check passes)
3	Valid admin authentication credentials / tokens available for all auth scenarios
4	Test data seeded: at least 1 large, 1 medium, and 1 minimal customer; 1 large and 1 minimal study
5	Pre-fix baseline response snapshots saved (for response equivalence tests TC-B-008)
7	Log aggregation is accessible for EF/serialization error verification

Exit Criteria (Pass)

#	Criterion	Priority
1	All Category B (Fix Validation) test cases pass — no response behavior changes for all 3 endpoints	Critical
2	Response equivalence confirmed for pre-fix vs fixed build (TC-B-008) — zero unacceptable differences	Critical
3	All Category D (Auth/AuthZ) test cases pass — auth behavior unchanged	Critical
4	Zero HTTP 500 errors observed for any input category (TC-E-001)	Critical
5	Zero EF query translation or serialization errors in logs (TC-E-004, TC-E-005)	Critical
6	Performance improvement validated — fixed build response times not worse than pre-fix; improvement documented for large inputs (TC-H-001 to TC-H-003)	High
7	AsyncNetworkIO metric reduced or not elevated in monitoring during burst (TC-H-006)	High
8	Memory/CPU trend stable — no growth or leak pattern during 15-min sustained run (TC-H-007, TC-H-008)	High
9	Concurrency tests pass — no 500s, no deadlocks, acceptable latency (TC-H-009, TC-H-010)	High
10	No new defects introduced in adjacent Admin API endpoints (TC-G-001)	Medium
11	Evidence package complete (response diffs, perf metrics, monitoring screenshots, log excerpts)	Medium

Exit Criteria (Fail / Block)

Any of the following automatically blocks promotion to the next environment:

- Any Critical criterion above is not met
- HTTP 500 returned for any test input
- Response schema or content differs from pre-fix baseline in a non-approved way

- Memory growth / leak pattern observed during sustained run
- New EF or serialization exceptions in logs

9. Risk & Mitigation

#	Risk	Likelihood	Impact	Mitigation Strategy
R-01	EF projection is not fully translatable to SQL — a sub-expression uses in-memory evaluation, causing <code>InvalidOperationException</code> at runtime (500 error)	Medium	High	1. Execute TC-E-004 early (log scan after first happy-path call) 2. Validate with SQL Profiler that generated SQL is a projected SELECT, not SELECT *. 3. Escalate immediately to dev if 500 observed.
R-02	Null/empty navigation handling differs in EF projection vs in-memory — empty studies list becomes null or vice versa, silently breaking downstream consumers	Medium	High	1. Execute TC-B-009 (0-study customer) and TC-F-002 (schema validation) explicitly. 2. Compare with pre-fix baseline; check Admin UI rendering (TC-G-002).
R-03	Performance improvement is not observable in lower environments (FB/Dev Master) due to small dataset or query plan caching — fix appears neutral when it's improved	Medium	Medium	1. Seed a sufficiently large customer in each env; also validate in Stage with realistic dataset (TC-H-007, TC-H-008). 2. Do not close performance validation on FB alone.
R-04	The <code>globalCustomerId</code> endpoint uses a separate code path that was not updated with the same EF projection fix	Low–Medium	High	1. Explicitly run all fix validation cases (Category B) against <code>/customer/{globalCustomerId}</code> separately. 2. Compare results with <code>customerId</code> endpoint (TC-B-004, TC-B-005). 3. Raise dev question Q4 in Section 2.
R-05	Pre-fix baseline responses not captured before deployment, making response equivalence test (TC-B-008) impossible	Low	Medium	1. Capture pre-fix snapshots as a mandatory entry criterion before deploying to each environment. 2. Use source control or a shared artifact store to preserve them.

10. Defect Reporting Template

Use this template when a test case fails during validation. Copy into a new JIRA defect and link it to EDC-42653.

Summary

[TC ID] [Brief description of what failed]. Example: "TC-B-008 — Response field 'studyCount' missing from GET `/api/v1/customer/{customerId}` response on fixed build"

JIRA Fields

Issue Type: Bug

Priority: [Critical / High / Medium / Low — based on impact]

Labels: EDC-42653, regression, admin-api, fix-validation

Epic Link: EDC-42653

Environment: [Feature Branch / Dev Master / Stage]
Build/Version: [Deployed build number or commit SHA]

Description

Test Case Failed: [TC ID — TC Description]
Test Category: [e.g., Fix Validation / Contract Testing / Performance]
Executed On: [Date and Environment]
Executed By: [QA Engineer name]

Steps to Reproduce

1. [Exact step 1]
2. [Exact step 2]
Request:
Method: GET
URL: [Full URL with path parameter]
Headers:
Authorization: Bearer <redacted>
Content-Type: application/json

Expected Result

[What the test case expected — reference the test case Expected Result column]

Actual Result

[What actually happened — exact status code, response body, error message, log output]

Evidence

[] Request/Response screenshot or Postman export attached
[] Log output (EF error / serialization error) attached
[] Pre-fix vs fixed build response diff attached (if applicable)
[] Monitoring screenshot (if perf/memory related) attached

Impact Assessment

- Endpoints affected: [List]
- Downstream consumers affected: [Admin UI / Other]
- Severity justification: [e.g., "Field missing from response breaks Admin UI rendering"]

Linked Issues

- Blocks: EDC-42653
- Related: EDC-42654

Appendix — Evidence Capture Checklist

Required before QA sign-off on each environment:

#	Evidence Item	Env: FB	Env: Dev Master	Env: Stage
1	Pre-fix vs fixed response diffs for each endpoint (large + small dataset)	☐	☐	☐
2	Response time measurement table (pre-fix vs fixed, per endpoint, per data size)	☐	☐	☐
3	Monitoring screenshot — CPU/memory trend during sustained run	—	☐	☐
4	Monitoring screenshot — AsyncNetworkIO metric during burst test	☐	☐	☐
5	Log excerpt — zero EF/serialization errors after full test run	☐	☐	☐
6	SQL Profiler / Query Store — confirmation of projected SQL (not SELECT *)	—	☐	☐
7	Concurrency test results (pass rates, latency)	—	☐	☐

End of Test Plan — EDC-42653 | Version 1.0